

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	D.Hemasree	Reg. No.:	192524114
Section / Batch:	2025	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT**Q1. Closest Pair Problem – City Planning System (Brute Force)****Scenario / Problem Interpretation**

A city planning system needs to compute the distances between all public service centers (fire stations, hospitals, police stations, schools, libraries, post offices) to identify which two centers are located closest to each other. This is a classical Closest-Pair of Points problem, solved here using the Brute Force approach.

a) How Brute-Force Closest Pair is Applied

- Represent each public service center as a coordinate point (x, y) on the city map.
- Generate every possible unique pair of points from the given set of n centers.
- For each pair, compute the Euclidean distance using the formula: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.
- Keep track of the minimum distance found so far and the pair of centers that produced it.
- After checking all pairs, the pair with the smallest distance is reported as the closest pair of service centers.
- This guarantees correctness because every possible pair is examined exhaustively — no pair is skipped.

b) Derivation of Time Complexity

For n points, the total number of unique pairs is given by combinations: $C(n, 2) = n(n-1)/2$.

Each pair requires a constant amount of work (a distance calculation and a comparison), so the total work done is proportional to $n(n-1)/2$, which simplifies to:

Time Complexity = $O(n^2)$

Space Complexity = $O(1)$ extra space (excluding input storage), since only the minimum distance and pair need to be tracked.

c) Performance Limitations

- Quadratic growth: as the number of service centers n increases, the number of comparisons grows as n^2 , making it very slow for large cities with thousands of centers.
- Redundant computation: many far-apart pairs are still compared even though they can never be the closest pair.
- Poor scalability: for real-time city planning systems with large datasets, Brute Force becomes impractical.
- Better alternative: the Divide-and-Conquer Closest Pair algorithm reduces the time complexity to $O(n \log n)$ by recursively splitting the point set and only checking a narrow strip near the dividing line.

Execution — Python Code

```
import itertools
import math

# Public service centers (x, y) coordinates on the city map
centers = {
    "A-FireStation": (2, 3),
    "B-Hospital": (12, 30),
    "C-PoliceStn": (40, 50),
    "D-Library": (5, 1),
    "E-School": (12, 10),
    "F-PostOffice": (3, 4)
}

def euclidean(p1, p2):
    return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)

pairs_checked = 0
min_dist = float('inf')
closest_pair = None
```

```

names = list(centers.keys())
for i, j in itertools.combinations(range(len(names)), 2):
    p1, p2 = centers[names[i]], centers[names[j]]
    d = euclidean(p1, p2)
    pairs_checked += 1
    print(f"Comparing {names[i]} {p1} - {names[j]} {p2} -> distance = {d:.3f}")
    if d < min_dist:
        min_dist = d
        closest_pair = (names[i], names[j])

print("\nTotal pairwise comparisons made:", pairs_checked)
print("Closest Pair of Service Centers:", closest_pair)
print("Minimum Distance:", round(min_dist, 3))

```

Output

```

Comparing A-FireStation (2, 3) - B-Hospital (12, 30) -> distance = 28.792
Comparing A-FireStation (2, 3) - C-PoliceStn (40, 50) -> distance = 60.440
Comparing A-FireStation (2, 3) - D-Library (5, 1) -> distance = 3.606
Comparing A-FireStation (2, 3) - E-School (12, 10) -> distance = 12.207
Comparing A-FireStation (2, 3) - F-PostOffice (3, 4) -> distance = 1.414
Comparing B-Hospital (12, 30) - C-PoliceStn (40, 50) -> distance = 34.409
Comparing B-Hospital (12, 30) - D-Library (5, 1) -> distance = 29.833
Comparing B-Hospital (12, 30) - E-School (12, 10) -> distance = 20.000
Comparing B-Hospital (12, 30) - F-PostOffice (3, 4) -> distance = 27.514
Comparing C-PoliceStn (40, 50) - D-Library (5, 1) -> distance = 60.216
Comparing C-PoliceStn (40, 50) - E-School (12, 10) -> distance = 48.826
Comparing C-PoliceStn (40, 50) - F-PostOffice (3, 4) -> distance = 59.034
Comparing D-Library (5, 1) - E-School (12, 10) -> distance = 11.402
Comparing D-Library (5, 1) - F-PostOffice (3, 4) -> distance = 3.606
Comparing E-School (12, 10) - F-PostOffice (3, 4) -> distance = 10.817

Total pairwise comparisons made: 15
Closest Pair of Service Centers: ('A-FireStation', 'F-PostOffice')
Minimum Distance: 1.414

```

Verification & Interpretation

With $n = 6$ centers, $C(6,2) = 15$ comparisons were performed, matching the theoretical $O(n^2)$ prediction. The closest pair identified is the Fire Station and Post Office, at a distance of 1.414 units, confirming the correctness of the exhaustive search. This result helps city planners identify overlapping service coverage and optimize placement of new facilities.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____